

M11 Learner Book

Storage & File Formats Foundations | Release Candidate 1 | 10 controlled lessons

M11-L01 - Row vs columnar storage; where Parquet fits

Learning objective

Explain why columnar storage improves many analytical scans.

Why this matters in Data Engineering

Storage format choices affect cost, scan volume and downstream tool behavior.

Understand

- CSV is row-oriented text; Parquet is a typed columnar binary format.
- Analytical queries often need a subset of columns.
- Compression/encoding can exploit similar values within columns.

Try / inspect

```
orders.csv -> orders.parquet  
query: select region, qty where status=Completed
```

Common failure

Calling Parquet a database or assuming it provides transactions by itself.

Your turn - independent proof

Explain why a two-column query can read less than a wide row file and state what Parquet does NOT provide.

Move on when

You can distinguish file format from table/database semantics.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L02 - Parquet schema, row groups and metadata

Learning objective

Inspect schema and metadata rather than treating a Parquet file as opaque.

Why this matters in Data Engineering

Correct schema and metadata make reads predictable and debuggable.

Understand

- A Parquet file stores typed columns plus metadata.
- Row groups divide rows into independently readable groups.
- Readers use metadata to locate needed column chunks.

Try / inspect

record rows, columns, schema, row-group count

Common failure

Memorizing one writer's row-group count as universal.

Your turn - independent proof

Create/inspect a Parquet file and record schema plus row/row-group evidence.

Move on when

You can explain metadata versus data pages.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L03 - Projection, predicates and analytical reads

Learning objective

Separate column projection from row filtering/predicate behavior.

Why this matters in Data Engineering

Efficient reads depend on asking for only needed data and understanding what can be skipped.

Understand

- Projection chooses columns.
- A predicate filters rows.
- Metadata/statistics can help some readers skip work; exact behavior is engine-dependent.

Try / inspect

```
read columns=[order_id, region, qty]
filter region == East
```

Common failure

Saying predicate pushdown always eliminates all unnecessary I/O.

Your turn - independent proof

Read only selected columns and filter a region; explain the two optimizations separately.

Move on when

You can describe what was requested and what evidence the engine exposes.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L04 - Compression, small files and compaction

Learning objective

Explain why many tiny files hurt metadata/open overhead and how compaction helps.

Why this matters in Data Engineering

Lakehouse workloads can fail operationally even when every file is valid.

Understand

- Compression reduces bytes but has CPU tradeoffs.
- Many tiny files increase listing/open/planning overhead.
- Compaction must preserve row identity/counts and be performed safely.

Try / inspect

```
12 tiny parquet files -> compacted.parquet  
reconcile row count before/after
```

Common failure

Compacting without reconciliation or assuming one giant file is always best.

Your turn - independent proof

Create many tiny files, compact them and prove row count is unchanged.

Move on when

You can explain small-file cost without inventing a magic target size.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L05 - Partitioning strategy and over-partitioning

Learning objective

Choose partition keys from query patterns and cardinality, not habit.

Why this matters in Data Engineering

Bad partitioning creates tiny directories/files and expensive metadata scans.

Understand

- Partitioning creates directory/data layout boundaries.
- Low/moderate-cardinality frequently filtered fields can be useful.
- High-cardinality keys often over-partition small/medium data.

Try / inspect

```
partition by order_date
inspect directory tree
```

Common failure

Partitioning by customer_id only because it is an important business key.

Your turn - independent proof

Partition the lab by date and explain why customer_id is a poor choice here.

Move on when

You can state query pattern, cardinality and size reasoning.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L06 - Lake vs warehouse vs lakehouse

Learning objective

Explain architectural roles instead of treating the terms as marketing labels.

Why this matters in Data Engineering

Storage architecture affects governance, table behavior and analytical access patterns.

Understand

- Data lake emphasizes flexible object/file storage.
- Warehouse emphasizes managed analytical tables/SQL governance.
- Lakehouse combines data-lake storage with table-management/transaction features.

Try / inspect

plain Parquet + metadata vs transactional table layer

Common failure

Assuming a folder of Parquet files is automatically a trustworthy lakehouse table.

Your turn - independent proof

List what plain Parquet lacks for safe concurrent updates/history.

Move on when

You can distinguish storage layer from table-management layer.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L07 - Delta tables, transaction log and ACID concepts

Learning objective

Explain how a transactional table format adds commit/history semantics over files.

Why this matters in Data Engineering

Reliable incremental pipelines need atomic table state, not manual file replacement.

Understand

- Delta stores data files plus a transaction log.
- Atomic commits expose consistent table versions.
- Do not manually edit/delete transaction-log files.

Try / inspect

table version -> active files -> commit actions

Common failure

Deleting log files or manually swapping Parquet files to simulate a transaction.

Your turn - independent proof

Create/inspect a local Delta table when runtime is available and identify commit/log actions.

Move on when

You can explain what makes a Delta table more than Parquet files.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L08 - MERGE/upsert and current-state records

Learning objective

Apply deterministic update/insert rules to current-state tables.

Why this matters in Data Engineering

Incremental pipelines need stable keys and version/winner rules.

Understand

- MERGE matches source and target by declared key.
- Updates must have deterministic newer/winner logic.
- Reconcile updated keys and total row counts after merge.

Try / inspect

08002 newer row updates qty; 08013 inserts

Common failure

MERGE without a stable key or version rule, causing duplicates/incorrect winners.

Your turn - independent proof

Run or design the merge and prove one update plus one insert with reconciled row count.

Move on when

You can defend key, update rule and rerun behavior.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L09 - Schema enforcement, controlled evolution and compatibility

Learning objective

Separate unexpected breaking drift from explicitly approved evolution.

Why this matters in Data Engineering

Silent schema change can corrupt downstream assumptions.

Understand

- Enforcement blocks incompatible writes.
- Evolution should be explicit and reviewed.
- Additive changes can be compatible while renames/type changes often need migration logic.

Try / inspect

strict append fails -> explicit schema evolution -> validate

Common failure

Automatically accepting every new column/type because the writer can.

Your turn - independent proof

Prove a strict mismatch is blocked, then apply an explicit approved evolution.

Move on when

You can state why the change is safe or breaking.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M11-L10 - Time travel, retention, VACUUM safety and medallion checkpoint

Learning objective

Use version history for audit/recovery and place transformations deliberately across Bronze/Silver/Gold.

Why this matters in Data Engineering

Recovery and layered architecture are part of operating trustworthy storage.

Understand

- Time travel reads prior table versions while retained files exist.
- VACUUM can remove old files and therefore historical access.
- Bronze preserves raw fidelity; Silver validates/cleans; Gold serves business models.
- Retention and compaction choices must respect recovery requirements.

Try / inspect

```
read version 0 and current  
Bronze -> Silver -> Gold
```

Common failure

Running VACUUM before required recovery evidence or mixing business aggregates into Bronze.

Your turn - independent proof

Build/defend the packaged medallion example and explain one retention/recovery tradeoff.

Move on when

You can defend file/table format, layer and recovery choices together.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.